CC451- Interacción Humano Computadora

Laboratorio Dirigido 06

Constelaciones Históricas 3D: Visualización Interactiva con D3.js, WebGL e Integración Programática de LLM

Tema: Interacción Programática con Modelos de Lenguaje (LLM) **Tecnologías:** D3.js, WebGL, Gemini API

Contenido

1. Introducción
2. Recursos Informáticos
3. Desarrollo
   - 3.1 Descripción del Proyecto
   - 3.2 Arquitectura de la Aplicación
   - 3.3 Estructura de Datos: Nodos Históricos
4. Ejercicio Guiado
   - 4.1 Configuración del Entorno
   - 4.2 Creación de la Espiral Temporal con D3.js
   - 4.3 Renderizado de Nodos Luminosos con WebGL
   - 4.4 Visualización de Conexiones Causales
   - 4.5 Integración Programática con el LLM (Gemini API)
   - 4.6 Sistema de Chat con Personajes Históricos
   - 4.7 Optimización de Prompts para Personajes
5. Tarea
   - 5.1 Requisitos de Entrega
   - 5.2 Rúbrica de Evaluación
6. Referencias

1. Introducción

Objetivo general:

Desarrollar una aplicación web interactiva que combine visualización de datos en 3D mediante D3.js y WebGL con interactividad impulsada por modelos de lenguaje (LLM). El estudiante aprenderá a crear interfaces visuales complejas donde cada elemento gráfico puede desencadenar una conversación contextual con inteligencia artificial, demostrando la integración programática entre frontend y servicios de IA generativa.

Objetivos específicos:

- Construir una visualización tridimensional tipo "constelación histórica" usando D3.js y WebGL donde eventos históricos representan nodos luminosos en una espiral temporal.
- Implementar conexiones causales entre nodos que se revelan mediante interacción de zoom.
- Integrar la API de Gemini para permitir conversaciones contextuales con personajes históricos representados por los nodos.
- Optimizar prompts del sistema para generar respuestas coherentes con la época y personalidad de cada personaje histórico.
- Comprender el flujo completo de datos: desde la interacción del usuario en WebGL hasta la generación de respuestas por el LLM y su presentación en la interfaz.

> ### Aplicación de ejemplo
>
> La aplicación a desarrollar es una **línea de tiempo en espiral 3D** (también denominada *mapa de constelaciones históricas*) donde cada evento es un nodo luminoso. Al acercar el zoom sobre un nodo, se revelan conexiones causales que vinculan épocas diferentes. Al hacer clic en un personaje histórico, el usuario puede mantener una conversación con él a través de un LLM, quien responde con el conocimiento, lenguaje y perspectiva propios de su época.

2. Recursos Informáticos

- **API de Gemini (Google AI):** https://ai.google.dev/
- **Google AI Studio:** https://aistudio.google.com/
- **Documentación de la API Gemini:** https://ai.google.dev/gemini-api/docs/
- **D3.js (Data-Driven Documents):** https://d3js.org/
- **Documentación de D3.js:** https://github.com/d3/d3/wiki
- **Three.js (WebGL):** https://threejs.org/
- **Node.js y npm:** https://nodejs.org/
- **Editor de código:** Visual Studio Code o similar
- **Navegador moderno:** Chrome, Firefox o Edge con soporte WebGL

3. Desarrollo

3.1 Descripción del Proyecto

El proyecto consiste en una aplicación web que visualiza eventos históricos como una **constelación tridimensional**. Cada evento o personaje histórico es representado por un **nodo luminoso** posicionado a lo largo de una espiral temporal. Los ejes de la espiral representan:

- **Eje X-Y:** La espiral bidimensional donde cada vuelta completa representa un siglo.
- **Eje Z (profundidad):** Categorías temáticas (Ciencia, Política, Arte, Filosofía, Tecnología).
- **Radio del nodo:** Importancia del evento en la escala histórica.
- **Color del nodo:** Categoría temática a la que pertenece.
- **Intensidad luminosa:** Impacto causal del evento en eventos

posteriores. La interactividad se estructura en tres niveles:

1. **Navegación orbital:** El usuario rota y hace zoom en la constelación 3D para explorar diferentes épocas.
2. **Zoom causal:** Al acercarse a un nodo, líneas curvas animadas revelan qué eventos posteriores fueron influenciados causalmente por este.
3. **Diálogo con personajes:** Al hacer clic en un nodo que representa a una figura histórica, se abre un panel de chat donde un LLM adopta la personalidad de esa figura y responde preguntas en contexto.

3.2 Arquitectura de la Aplicación

La arquitectura sigue un patrón de capas:

Tabla 1 Componentes de la arquitectura

Capa	Tecnología	Función
Visualización 3D	Three.js + D3.js	Renderizado de la espiral, nodos luminosos y conexiones
Interactividad	D3.js Zoom + Raycaster	Navegación orbital, detección de clic en nodos
Datos históricos	JSON estático / API	Información de eventos, personajes y relaciones causales
Integración LLM	Google Gemini API	Generación de respuestas contextualizadas por personaje
Interfaz de chat	HTML/CSS/JS	Panel conversacional con histórico de mensajes
Gestiónde prompts	JavaScript	Construcción dinámica de system instructions por personaje

3.3 Estructura de Datos: Nodos Históricos

1.

```
          prompt_personaje
{
  "id": "newton-1687",
  "nombre": "Isaac Newton",
  "año": 1687,
  "categoria": "ciencia",
  "epoca": "revolucion_cientifica",
  "importancia": 10,
  "posicion": { "espiral": 5.2, "z": 1 },
  "resumen": "Publicación de los Principia Mathematica",
  "conexiones": ["illuminismo-1750", "einstein-1905"],
  "prompt_personaje": "Eres Isaac Newton en 1687. Has acabado de...",
  "avatar": "./assets/newton.png",
  "metadata":
    { "nacionalidad":
    "Inglesa",
    "obra_principal": "Philosophiæ Naturalis Principia Mathematica",
    "contribucion": "Leyes del movimiento y gravitación universal"
  }
}
```

Cada nodo en la constelación sigue un esquema de datos estructurado: El campo es fundamental: contiene las **system instructions** que se enviarán al LLM para que adopte la personalidad del personaje. Este campo se utiliza para construir dinámicamente el contexto de la conversación cuando el usuario interactúa con el nodo.

4. Ejercicio Guiado

4.1 Configuración del Entorno

Paso 1: Crear estructura de directorios

Cree una carpeta para el proyecto con la siguiente estructura:

```
constelaciones-historicas/
├── index.html
├── css/
│   └── styles.css
├── js/
│   ├── main.js
│   ├── constellation.js
│   ├── llm-client.js
│   └── data.js
├── assets/
│   └── (imágenes de avatares)
└── README.md
```

Paso 2: Obtener API Key de Gemini

1. Visite Google AI Studio.

2. Genere una clave de API en la sección "Get API key".

3. Almacene la clave de forma segura; se usará en las llamadas a la API.

Importante: Seguridad de la API Key

Para el desarrollo local puede incluir la API key en un archivo `config.js` que no se suba al repositorio (agregar a `.gitignore`). Para producción, utilice variables de entorno o un backend proxy.

42 Creación de la Espiral Temporal con D3.js

Paso 3: Implementar la espiral temporal

En el archivo `constellation.js` , cree la geometría de la espiral:

```
// constellation.js - Geometría de la espiral temporal
import * as d3 from 'd3';
import * as THREE from 'three';

/**
 * Genera puntos a lo largo de una espiral logarítmica 3D.
 * Cada vuelta representa un siglo, el eje Z separa categorías.
 */
export class ConstellationSpiral {
  constructor(containerId, width, height) {
    this.scene = new THREE.Scene();
    this.camera = new THREE.PerspectiveCamera(60, width/height, 0.1, 1000);
    this.renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
    this.renderer.setSize(width, height);
    this.renderer.setPixelRatio(window.devicePixelRatio);
    document.getElementById(containerId).appendChild(this.renderer.domElement);

    this.camera.position.set(0, 30, 60);
    this.camera.lookAt(0, 0, 0);

    this.nodes = [];
    this.connections = [];
    this.raycaster = new THREE.Raycaster();
    this.mouse = new THREE.Vector2();

    this.setupLighting();
    this.setupControls();
  }

  setupLighting() {
    const ambient = new THREE.AmbientLight(0x404040, 0.5);
    this.scene.add(ambient);

    const pointLight = new THREE.PointLight(0xffffff, 1, 100);
    pointLight.position.set(0, 20, 0);
    this.scene.add(pointLight);
  }

  setupControls() {
    // OrbitControls para navegación 3D
    // npm install three/examples/jsm/controls/OrbitControls
  }

  /**
   * Convierte coordenadas históricas (año, categoría) a posición 3D.
   * a: parámetro de la espiral, b: factor de crecimiento
   */
  historicalTo3D(year, categoryIndex, importance) {
    const centuriesFromOrigin = (year - 1500) / 100;
    const angle = centuriesFromOrigin * 2 * Math.PI;
    const a = 2;   // Radio inicial
    const b = 0.3; // Factor de crecimiento por vuelta
    const r = a * Math.exp(b * centuriesFromOrigin);

    const x = r * Math.cos(angle);
    const y = r * Math.sin(angle);
```

```
    const z = categoryIndex * 8 - 20; // Separación por categoría

    return new THREE.Vector3(x, y, z);
  }

  addNode(eventData) {
    const pos = this.historicalTo3D(
      eventData.año,
      this.categoryIndex(eventData.categoria),
      eventData.importancia
    );

    // Geometría del nodo: esfera con emisión luminosa
    const geometry = new THREE.SphereGeometry(
      0.5 + eventData.importancia * 0.15, 32, 32
    );
    const material = new THREE.MeshPhongMaterial({
      color: this.categoryColor(eventData.categoria),
      emissive: this.categoryColor(eventData.categoria),
      emissiveIntensity: 0.6,
      transparent: true,
      opacity: 0.9
    });

    const mesh = new THREE.Mesh(geometry, material);
    mesh.position.copy(pos);
    mesh.userData = eventData; // Almacenar datos para interacción

    this.scene.add(mesh);
    this.nodes.push(mesh);

    // Halo luminoso alrededor del nodo
    this.addGlow(mesh, eventData.categoria);

    return mesh;
  }

  addGlow(mesh, category) {
    const glowGeometry = new THREE.SphereGeometry(
      mesh.geometry.parameters.radius * 1.5, 32, 32
    );
    const glowMaterial = new THREE.MeshBasicMaterial({
      color: this.categoryColor(category),
      transparent: true,
      opacity: 0.15
    });
    const glow = new THREE.Mesh(glowGeometry, glowMaterial);
    mesh.add(glow); // Hijo del nodo, se mueve con él
  }

  categoryColor(category) {
    const colors = {
      ciencia: 0x3498db,
      política: 0xe74c3c,
      arte: 0xf39c12,
      filosofia: 0x9b59b6,
      tecnologia: 0x2ecc71
    };
    return colors[category] || 0xffffff;
  }

  categoryIndex(category) {
    const categories = ['ciencia', 'filosofia', 'arte', 'politica', 'tecnologia'];
    return categories.indexOf(category);
  }
}
```

4.3 Renderizado de Nodos Luminosos con WebGL

Paso 4: Implementar efectos de luminosidad WebGL

Amplíe la clase anterior con efectos visuales avanzados usando shaders personalizados:

```
// Shaders para efecto de pulso luminoso
const vertexShader = `
  varying vec3 vNormal;
  varying vec3 vPosition;
  void main() {
    vNormal = normalize(normalMatrix * normal);
    vPosition = position;
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
  }
`;

const fragmentShader = `
  uniform float time;
  uniform vec3 baseColor;
  uniform float intensity;
  varying vec3 vNormal;
  varying vec3 vPosition;

  void main() {
    // Efecto de pulso sinusoidal
    float pulse = sin(time * 2.0) * 0.1 + 0.9;

    // Gradiente basado en la normal (efecto 3D)
    float glow = pow(0.7 - dot(vNormal, vec3(0, 0, 1.0)), 2.0);

    vec3 color = baseColor * intensity * pulse;
    color += baseColor * glow * 0.5;

    gl_FragColor = vec4(color, 0.9);
  }
`;

// En la clase ConstellationSpiral, reemplace el material estándar:
addNodeWithShader(eventData) {
  const pos = this.historicalTo3D(eventData.año,
    this.categoryIndex(eventData.categoria), eventData.importancia);

  const geometry = new THREE.SphereGeometry(
    0.5 + eventData.importancia * 0.15, 32, 32);

  const material = new THREE.ShaderMaterial({
    vertexShader: vertexShader,
    fragmentShader: fragmentShader,
    uniforms: {
      time: { value: 0 },
      baseColor: { value: new THREE.Color(this.categoryColor(eventData.categoria)) },
      intensity: { value: 1.0 + eventData.importancia * 0.1 }
    },
    transparent: true
  });

  const mesh = new THREE.Mesh(geometry, material);
  mesh.position.copy(pos);
  mesh.userData = { ...eventData, shaderUniforms: material.uniforms };
```

```
  this.scene.add(mesh);
  this.nodes.push(mesh);

  return mesh;
}

// En el loop de animación, actualizar el tiempo:
animate() {
  requestAnimationFrame(() => this.animate());

  const time = performance.now() * 0.001;
  this.nodes.forEach(node => {
    if (node.userData.shaderUniforms) {
      node.userData.shaderUniforms.time.value = time;
    }
  });

  this.renderer.render(this.scene, this.camera);
}
```

4.4 Visualización de Conexiones Causales

Paso 5: Dibujar conexiones causales entre nodos

Las conexiones representan influencia histórica entre eventos:

```
// Agregar conexiones causales entre nodos
addCausalConnections() {
  this.nodes.forEach(sourceNode => {
    const sourceData = sourceNode.userData;
    if (!sourceData.conexiones) return;

    sourceData.conexiones.forEach(targetId => {
      const targetNode = this.nodes.find(n => n.userData.id === targetId);
      if (!targetNode) return;

      // Crear curva Bezier entre nodos
      const curve = this.createConnectionCurve(sourceNode, targetNode);
      this.scene.add(curve);

      this.connections.push({
        source: sourceNode,
        target: targetNode,
        mesh: curve,
        originalOpacity: 0.0 // Invisible hasta zoom cercano
      });
    });
  });
}

createConnectionCurve(nodeA, nodeB) {
  const start = nodeA.position.clone();
  const end = nodeB.position.clone();

  // Punto de control para curva elevada
  const mid = new THREE.Vector3().addVectors(start, end).multiplyScalar(0.5);
  mid.z += 5; // Arco hacia arriba

  const curve = new THREE.QuadraticBezierCurve3(start, mid, end);
  const points = curve.getPoints(50);
  const geometry = new THREE.BufferGeometry().setFromPoints(points);

  const material = new THREE.LineBasicMaterial({
    color: 0xffffff,
    transparent: true,
    opacity: 0.0, // Inicialmente invisible
    linewidth: 1
  });

  return new THREE.Line(geometry, material);
}

// Revelar conexiones según la distancia de la cámara
updateConnectionsByZoom() {
  const cameraDistance = this.camera.position.length();

  this.connections.forEach(conn => {
    const distance = conn.source.position.distanceTo(this.camera.position);

    // Mostrar conexiones cuando la cámara está cerca del nodo origen
    if (distance < 25) {
      conn.mesh.material.opacity = Math.min(1.0, (25 - distance) / 10);
    } else {
      conn.mesh.material.opacity = 0.0;
    }
  });
}
```

4.5 Integración Programática con el LLM (Gemini API)

Paso 6: Crear el cliente LLM

El archivo `llm-client.js` encapsula toda la comunicación con la API de Gemini:

```
// llm-client.js - Cliente para la API de Gemini
export class LLMClient {
  constructor(apiKey, modelName = 'gemini-2.0-flash') {
    this.apiKey = apiKey;
    this.modelName = modelName;
    this.baseUrl = 'https://generativelanguage.googleapis.com/v1beta/models';
    this.conversationHistory = []; // Memoria de la conversación
  }

  /**
   * Envía un mensaje al LLM con system instructions dinámicas.
   * El systemPrompt define la personalidad del personaje histórico.
   */
  async chat(message, systemPrompt, temperature = 0.7) {
    const url = \`\${this.baseUrl}/\${this.modelName}:generateContent?key=\${this.apiKey}
\`;

    // Construir el cuerpo de la solicitud con el system prompt
    const requestBody = {
      systemInstruction: {
        parts: [{ text: systemPrompt }]
      },
      contents: [
        ...this.conversationHistory,
        {
          role: 'user',
          parts: [{ text: message }]
        }
      ],
      generationConfig: {
        temperature: temperature,
        maxOutputTokens: 512,
        topP: 0.9
      }
    };

    try {
      const response = await fetch(url, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(requestBody)
      });

      if (!response.ok) {
        const error = await response.json();
        throw new Error(\`Error API: \${error.error?.message || response.statusText}\`);
      }

      const data = await response.json();
      const reply = data.candidates[0].content.parts[0].text;

      // Guardar en el historial para contexto
      this.conversationHistory.push(
        { role: 'user', parts: [{ text: message }] },
        { role: 'model', parts: [{ text: reply }] }
      );

      // Limitar historial a últimos 10 mensajes para no exceder tokens
      if (this.conversationHistory.length > 20) {
        this.conversationHistory = this.conversationHistory.slice(-20);
      }

      return reply;
    } catch (error) {
      console.error('Error en LLMClient.chat:', error);
      throw error;
    }
  }

  /**
```

```
   * Limpia el historial de conversación.
   * Útil al cambiar de personaje.
   */
  clearHistory() {
    this.conversationHistory = [];
  }

  /**
   * Configura el nivel de creatividad de las respuestas.
   * Útil para experimentar con diferentes temperaturas.
   */
  setTemperature(value) {
    this.temperature = Math.max(0, Math.min(2, value));
  }
}
```

4.6 Sistema de Chat con Personajes Históricos

Paso 7: Conectar la interacción 3D con el chat

En `main.js` , integre el raycaster con el panel de chat:

```
// main.js - Integración de visualización y LLM
import { ConstellationSpiral } from './constellation.js';
import { LLMClient } from './llm-client.js';
import { historicalData } from './data.js';
import { API_KEY } from './config.js';

class HistoricalConstellationsApp {
  constructor() {
    this.constellation = new ConstellationSpiral('canvas-container',
      window.innerWidth, window.innerHeight);
    this.llm = new LLMClient(API_KEY);
    this.currentCharacter = null;

    this.initData();
    this.setupInteraction();
    this.setupChatUI();
    this.animate();
  }

  initData() {
    // Cargar nodos desde datos
    historicalData.events.forEach(event => {
      this.constellation.addNodeWithShader(event);
    });
    this.constellation.addCausalConnections();
  }

  setupInteraction() {
    const canvas = this.constellation.renderer.domElement;

    canvas.addEventListener('click', (event) => {
      // Calcular posición normalizada del mouse
      const rect = canvas.getBoundingClientRect();
      const mouse = new THREE.Vector2(
        ((event.clientX - rect.left) / rect.width) * 2 - 1,
        -((event.clientY - rect.top) / rect.height) * 2 + 1
      );

      // Raycasting para detectar nodo clickeado
      const raycaster = new THREE.Raycaster();
      raycaster.setFromCamera(mouse, this.constellation.camera);

      const intersects = raycaster.intersectObjects(this.constellation.nodes);

      if (intersects.length > 0) {
        const selectedNode = intersects[0].object;
        this.onNodeClick(selectedNode.userData);
      }
    });

    // Cursor pointer al pasar sobre nodos
    canvas.addEventListener('mousemove', (event) => {
      const rect = canvas.getBoundingClientRect();
      const mouse = new THREE.Vector2(
        ((event.clientX - rect.left) / rect.width) * 2 - 1,
        -((event.clientY - rect.top) / rect.height) * 2 + 1
      );
      const raycaster = new THREE.Raycaster();
      raycaster.setFromCamera(mouse, this.constellation.camera);
      const intersects = raycaster.intersectObjects(this.constellation.nodes);
      canvas.style.cursor = intersects.length > 0 ? 'pointer' : 'default';
    });
  }

  onNodeClick(nodeData) {
    // Si el nodo tiene un personaje asociado, abrir chat
    if (nodeData.prompt_personaje) {
      this.openCharacterChat(nodeData);
    } else {
      // Mostrar información del evento
```

```
      this.showEventInfo(nodeData);
    }
  }

  openCharacterChat(characterData) {
    this.currentCharacter = characterData;
    this.llm.clearHistory(); // Nueva conversación por personaje

    const chatPanel = document.getElementById('chat-panel');
    const avatar = document.getElementById('chat-avatar');
    const name = document.getElementById('chat-name');
    const messages = document.getElementById('chat-messages');

    avatar.src = characterData.avatar;
    avatar.alt = characterData.nombre;
    name.textContent = \`\${characterData.nombre} (\${characterData.año})\`;
    messages.innerHTML = \`


        Estás conversando con \${characterData.nombre}.
        \${characterData.resumen}


    \`;

    chatPanel.classList.add('active');

    // Mensaje de bienvenida del personaje
    this.sendToLLM("Presentate brevemente a quien te visita.",
characterData.prompt_personaje);
  }

  async sendToLLM(message, systemPrompt) {
    const messagesContainer = document.getElementById('chat-messages');

    // Mostrar indicador de carga
    const loadingDiv = document.createElement('div');
    loadingDiv.className = 'message loading';
    loadingDiv.textContent = 'Escribiendo...';
    messagesContainer.appendChild(loadingDiv);
    messagesContainer.scrollTop = messagesContainer.scrollHeight;

    try {
      const response = await this.llm.chat(message, systemPrompt);

      // Reemplazar indicador con respuesta
      loadingDiv.remove();
      this.addMessageToChat('character', response);
    } catch (error) {
      loadingDiv.remove();
      this.addMessageToChat('error', 'Error al comunicarse con el personaje. Intente
nuevamente.');
    }
  }

  addMessageToChat(sender, text) {
    const messagesContainer = document.getElementById('chat-messages');
    const msgDiv = document.createElement('div');
    msgDiv.className = \`message \${sender}\`;
    msgDiv.textContent = text;
    messagesContainer.appendChild(msgDiv);
    messagesContainer.scrollTop = messagesContainer.scrollHeight;
  }

  setupChatUI() {
    const input = document.getElementById('chat-input');
    const sendBtn = document.getElementById('chat-send');
    const closeBtn = document.getElementById('chat-close');

    const sendMessage = () => {
      const text = input.value.trim();
```

```
      if (!text || !this.currentCharacter) return;

      this.addMessageToChat('user', text);
      input.value = '';

      this.sendToLLM(text, this.currentCharacter.prompt_personaje);
    };

    sendBtn.addEventListener('click', sendMessage);
    input.addEventListener('keypress', (e) => {
      if (e.key === 'Enter') sendMessage();
    });

    closeBtn.addEventListener('click', () => {
      document.getElementById('chat-panel').classList.remove('active');
      this.currentCharacter = null;
    });
  }

  animate() {
    requestAnimationFrame(() => this.animate());
    this.constellation.updateConnectionsByZoom();
    this.constellation.animate();
  }
}

// Inicializar cuando el DOM esté listo
document.addEventListener('DOMContentLoaded', () => {
  new HistoricalConstellationsApp();
});
```

4.7 Optimización de Prompts para Personajes

Paso 8: Diseñar prompts de sistema efectivos

El archivo `data.js` contiene los datos y los prompts optimizados:

```
// data.js - Datos históricos y prompts de personajes
export const historicalData = {
  events: [
    {
      id: "newton-1687",
      nombre: "Isaac Newton",
      año: 1687,
      categoria: "ciencia",
      epoca: "revolucion_cientifica",
      importancia: 10,
      posicion: { espiral: 5.2, z: 1 },
      resumen: "Publicó los Principia Mathematica, estableciendo las leyes del movimiento
y la gravitación universal.",
      conexiones: ["illuminismo-1750", "einstein-1905"],
      prompt_personaje: \`Eres Sir Isaac Newton en el apogeo de tu carrera, año 1687.
Acabas de publicar "Philosophiæ Naturalis Principia Mathematica".

REGLAS DE COMPORTAMIENTO:
- Responde con el conocimiento científico disponible en tu época (finales del siglo XVII).
- No conoces conceptos posteriores a 1687: relatividad, mecánica cuántica, evolución,
electricidad.
- Eres meticuloso, profundamente religioso y crees que el universo sigue leyes matemáticas
divinas.
- Hablas con formalidad propia de un erudito inglés del siglo XVII.
- Menciona tu trabajo en óptica, matemáticas o alquimia cuando sea relevante.
- Si te preguntan sobre tecnologías futuras, responde con curiosidad hipotética, no con
conocimiento.
- Usa occasionalmente frases latinas académicas.

ESTILO: Preciso, filosófico, con humildad intelectual mezclada con confianza en el método
matemático.\`,
      avatar: "./assets/newton.png",
      metadata: {
        nacionalidad: "Inglesa",
        obra_principal: "Principia Mathematica",
        contribucion: "Leyes del movimiento y gravitación universal"
      }
    },
    {
      id: "leonardo-1510",
      nombre: "Leonardo da Vinci",
      año: 1510,
      categoria: "arte",
      epoca: "renacimiento",
      importancia: 9,
      posicion: { espiral: 3.1, z: 2 },
      resumen: "Polímata renacentista: pintor, anatomista, inventor e ingeniero.",
      conexiones: ["galileo-1610", "newton-1687"],
      prompt_personaje: \`Eres Leonardo da Vinci en 1510, trabajando en Milán bajo el
patrocinio de Ludovico Sforza.

REGLAS DE COMPORTAMIENTO:
- Eres un polímata: pintor, escultor, arquitecto, músico, matemático, ingeniero,
anatomista.
- Escribes al revés (espejo) pero en esta conversación usa texto normal por claridad.
- Eres obsesivo con la observación de la naturaleza; todo fenómeno natural te fascina.
- Tienes conocimiento anatómico avanzado para tu época por disecciones humanas.
- Diseñas máquinas voladoras, puentes móviles, armas y dispositivos mecánicos.
- No conoces la fotografía, electricidad, o máquinas de vapor.
- Eres paciente, curioso, y crees que "la pintura es cosa mental".
- Cuando describes, lo haces con detalle visual y anatómico.

ESTILO: Observador minucioso, mezcla de arte y ciencia, italiano renacentisto con
referencias a la natura.\`,
      avatar: "./assets/leonardo.png",
      metadata: {
        nacionalidad: "Italiana",
        obra_principal: "La Gioconda, La Última Cena",
        contribucion: "Arte renacentista, anatomía, ingeniería"
      }
    },
```

```
    {
      id: "einstein-1905",
      nombre: "Albert Einstein",
      año: 1905,
      categoria: "ciencia",
      epoca: "fisica_moderna",
      importancia: 10,
      posicion: { espiral: 8.05, z: 1 },
      resumen: "Año milagroso: publicó sobre el efecto fotoeléctrico, el movimiento
browniano, la relatividad especial.",
      conexiones: [" Oppenheimer-1945"],
      prompt_personaje: \`Eres Albert Einstein en 1905, el "Annus Mirabilis". Trabajas en
la Oficina de Patentes de Berna.

REGLAS DE COMPORTAMIENTO:
- Acabas de publicar cuatro artículos revolucionarios sobre el efecto fotoeléctrico,
  el movimiento browniano, la relatividad especial y la equivalencia masa-energía (E=mc²).
- No has desarrollado la relatividad general todavía (es 1915); no hables de curvatura del
espacio-tiempo.
- Eres joven (26 años), apasionado, con sentido del humor y amor por la música (violín).
- Crees profundamente en el determinismo científico pero no en las "acciones a distancia".
- No conoces la mecánica cuántica en su forma completa (eso viene después).
- Piensas en experimentos mentales (Gedankenexperimente) para explicar conceptos.
- Eres de origen judío-alemán, hablas con acento germánico conceptual.

ESTILO: Pensamiento visual, imaginativo, con humor modesto, explica con analogías
cotidianas.\`,
      avatar: "./assets/einstein.png",
      metadata: {
        nacionalidad: "Alemana (posteriormente estadounidense)",
        obra_principal: "Artículos del Annus Mirabilis 1905",
        contribucion: "Relatividad especial, efecto fotoeléctrico"
      }
    },
    {
      id: "galileo-1610",
      nombre: "Galileo Galilei",
      año: 1610,
      categoria: "ciencia",
      epoca: "revolucion_cientifica",
      importancia: 9,
      posicion: { espiral: 4.1, z: 1 },
      resumen: "Publicó 'Sidereus Nuncius' con observaciones telescópicas de Júpiter y la
Luna.",
      conexiones: ["newton-1687"],
      prompt_personaje: \`Eres Galileo Galilei en 1610, tras publicar "Sidereus Nuncius".
Has observado las lunas de Júpiter, las montañas de la Luna y las fases de Venus.

REGLAS DE COMPORTAMIENTO:
- Defiendes el heliocentrismo con pasión basada en evidencia observacional.
- Eres enemigo de seguir autoridades antiguas (Aristóteles) sin experimentar.
- "La filosofía está escrita en ese gran libro que tenemos abierto ante los ojos: el
universo".
- No conoces la gravitación universal (eso es Newton después); tus leyes son cinemáticas.
- Eres católico pero en conflicto con la Iglesia por tus ideas.
- Hablas italiano, a veces latín académico.
- Te apasiona el diálogo como método de enseñanza.

ESTILO: Didáctico, combativo intelectualmente, defensor de la evidencia empírica.\`,
      avatar: "./assets/galileo.png",
      metadata: {
        nacionalidad: "Italiana",
        obra_principal: "Sidereus Nuncius, Diálogo sobre los dos máximos sistemas",
        contribucion: "Astronomía telescópica, cinemática"
      }
    },
    {
      id: "curie-1903",
      nombre: "Marie Curie",
      año: 1903,
      categoria: "ciencia",
      epoca: "fisica_moderna",
```

```
      importancia: 9,
      posicion: { espiral: 8.03, z: 1 },
      resumen: "Primera mujer en ganar un Nobel (Física, 1903) por investigación sobre
radiactividad.",
      conexiones: ["einstein-1905"],
      prompt_personaje: \`Eres Marie Skłodowska-Curie en 1903, acabas de ganar el Premio
Nobel de Física
compartido con Pierre Curie y Henri Becquerel.

REGLAS DE COMPORTAMIENTO:
- Eres la primera mujer en ganar un Nobel; enfrentaste enormes barreras por tu género.
- Eres polaca de nacimiento, francesa por adopción; hablas con elegancia europea.
- Trabajas con radio y polonio; entiendes la radiactividad mejor que nadie en el mundo.
- No conoces los peligros completos de la radiación; trabajas sin protección.
- Eres madre, científica, y defensora de la educación para las mujeres.
- Eres meticulosa en la experimentación; extraes radio de toneladas de pechblenda.
- Pierre está vivo todavía (muere en 1906).

ESTILO: Determinada, precisa científicamente, con pasión contenida, defensora del
conocimiento universal.\`,
      avatar: "./assets/curie.png",
      metadata: {
        nacionalidad: "Polaca-Francesa",
        obra_principal: "Investigación sobre la radiactividad",
        contribucion: "Descubrimiento del radio y polonio, primer Nobel para una mujer"
      }
    }
  ]
};
```

> **Principios de optimización de prompts aplicados**
>
> **1. Contexto temporal estricto:** Cada prompt define explícitamente la fecha límite de conocimiento del personaje, evitando anacronismos.
>
> **2. Reglas de comportamiento numeradas:** Estructura clara que guía consistentemente las respuestas.
>
> **3. Estilo de comunicación definido:** Instrucciones sobre tono, vocabulario y referencias culturales propias de la época.
>
> **4. Límites de conocimiento:** Enumeración explícita de conceptos que el personaje NO debe conocer.
>
> **5. Personalidad distintiva:** Rasgos característicos que hacen única la voz de

5. Tarea

5.1 Requisitos de Entrega

Desarrolle una aplicación de **Constelaciones 3D relativas al tema de su proyecto** cumpliendo los siguientes requisitos:

Tarea 1: Visualización 3D (30%)

1. Implemente una espiral temporal 3D con al menos **10 nodos** distribuidos en múltiples categorías .
2. Cada nodo debe tener un efecto de luminosidad (emissive material o shader).
3. Implemente navegación orbital con zoom, rotación y paneo.
4. Agregue etiquetas o tooltips que muestren informacion relevante del tema al pasar el mouse.

Tarea 2: Conexiones Causales (20%)

1. Defina al menos **8 conexiones causales** entre nodos en el archivo de datos.
2. Implemente la visualización de conexiones que se revelan con el zoom (más cercano = más visible).
3. Use curvas Bezier o líneas con animación para las conexiones.

4. Agregue un modo alternativo donde todas las conexiones se muestren simultáneamente (botón toggle).

Tarea 3: Integración Programática con LLM (30%)

1. Implemente la comunicación con la API de Gemini usando la clase `LLMClient` .
2. Cree **al menos 5 personajes relativos a su tema de proyecto** con prompts de sistema optimizados que incluyan:
   - Contexto estricto (límite de conocimiento)
   - Reglas de comportamiento (mínimo 5 por personaje)
   - Estilo de comunicación definido
   - Enumeración de conceptos que NO debe conocer
3. Implemente el panel de chat funcional con historial de mensajes.
4. El chat debe mantener contexto de la conversación (memoria de mensajes anteriores).

Tarea 4: Optimización y Experimentación (20%)

1. Experimente con diferentes valores de **temperature** (0.0, 0.5, 1.0, 1.5) para un mismo personaje y prompt. Documente en una tabla cómo cambian las respuestas.
2. Cree una versión mejorada de uno de los prompts originales y explique qué cambios hizo y por qué.
3. Implemente una característica adicional a elección:
   - Búsqueda de nodos por alguna caracteristica util
   - Filtro por categoría temática
   - Generación de audio con TTS para las respuestas del personaje

ENTREGA:

Archivo comprimido con lo siguiente: Carpeta de proyecto con todos los archivos fuente.

Archivo README.md con Descripcion de la aplicación e instrucciones para ejecutar

Carpeta screenshots con capturas de pantalla de la aplicación funcionando

archivo CC451-2026-2-LAB06-NOMBRE-APELLIDO.pdf

5.2 Rúbrica de Evaluación

Tabla 2 Rúbrica de evaluación del laboratorio

Criterio	Excelente (4)	Satisfactorio (3)	En desarrollo (2)	Insuficiente (1)
Visualización 3D	Espiralfuncional, 10+ nodos con shaders/luz, navegación fluida, tooltips	Espiral con 8-9 nodos, materiales básicos, navegación funcional	Espiral incompleta, menos de 8 nodos, navegación limitada	Visualización plana o menos de 5 nodos
Conexiones causales	8+ conexiones con zoom reveal, curvas animadas, modo toggle	6-7 conexiones visibles, zoom reveal básico	4-5 conexiones, siempre visibles	Menos de 4 conexiones o ausentes
Integración LLM	5+ personajes con prompts optimizados, chat con memoria, manejo de errores	4 personajes con prompts básicos, chat funcional	2-3 personajes, prompts simples, chat parcial	1 personaje o integración no funcional
Optimización	Tabla de temperaturas completa, prompt mejorado documentado, feature adicional innovadora	Tabla de temperaturas básica, mejoras menores al prompt, feature funcional	Solo tabla de temperaturas o solo feature adicional	No experimentó ni agregó features
Documentación	README completo, informe detallado, screenshots claras	README e informe presentes pero breves	README o informe incompleto	Sin documentación

6. Referencias

- Google. (2025). *Gemini API Documentation*. Recuperado de https://ai.google.dev/gemi ni-api/docs

- Bostock, M., Ogievetsky, V., & Heer, J. (2011). D3: Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12), 2301-2309.
- Three.js Documentation. (2025). *Three.js - JavaScript 3D Library*. Recuperado de https://threejs.org/docs/
- Google AI Studio. (2025). *Prompt Engineering Guide*. Recuperado de https://ai.google. dev/gemini-api/docs/prompting-intro
- Murray, S. (2017). *Interactive Data Visualization for the Web* (2nd ed.). O'Reilly Media.